

JavaScript Interview Study Guide

Nine topic areas, discovered from current interview-report sources rather than a fixed syllabus — spanning scope and closures, this-binding, prototypal inheritance, coercion, the browser and Node.js event loops, DOM/practical coding patterns, V8 performance internals, and JS-specific security. Runtime claims are checked directly against MDN, the Node.js docs, and V8-internals writing.

68 questions · Basic → Intermediate → Advanced

9 topic areas

Output-prediction traces · runnable snippets · polyfills

Sourced from MDN, Node.js docs, GfG, Glassdoor, lydiahallie/javascript-questions

Updated Sep 2026

WHO THIS IS FOR

Intern

Junior

Mid

Senior

- **Intern/Junior** — expect scope, hoisting, this, and coercion output-prediction snippets (Sections 1–4).
- **Mid-level** — expect you to trace the event loop precisely and write debounce/throttle/deep-clone live (Sections 5, 7).
- **Senior/Staff** — expect V8-internals reasoning, Node runtime internals, and real security trade-offs (Sections 6, 8–9).

WHAT INTERVIEWERS EVALUATE

- **Runtime reasoning** — tracing execution order precisely, not just naming "the event loop."
- **Debugging instinct** — spotting closures-over-shared-state and stale this bugs on sight.
- **Code quality** — a debounce/throttle implementation that actually handles edge cases, not just the happy path.
- **Precise terminology** — microtask vs macrotask, reference vs value, said correctly and used correctly.

01 Scope, Hoisting & Closures

Sources are blunt about this one: closures are the single most-asked conceptual JS question, and 2026-era interviews want you to *use* one under pressure, not recite a definition.

1.1 BASIC COMPARISON var vs let vs const.

	VAR	LET	CONST
Scope	Function-scoped	Block-scoped	Block-scoped
Hoisting	Hoisted, initialized to undefined	Hoisted, but in the TDZ until declared (1.5)	Hoisted, in the TDZ
Reassignment	Allowed	Allowed	Not allowed (binding is constant, not the value — 3.6)
Redeclaration	Allowed	Not allowed	Not allowed

let/const (ES6) were introduced specifically to fix var's function-scoping surprises — block scope matches how every other C-family language behaves, which is exactly why 1.4's classic loop trap only exists with var.

function scope block scope

1.2 BASIC OUTPUT What does this print, and why?

```
console.log(a); // ?
var a = 5;

console.log(b); // ?
let b = 5;
```

The first line prints **undefined**. **Hoisting** moves var declarations (not their assignments) to the top of their enclosing function/global scope during compilation — so by the time `console.log(a)` runs, `a` already exists, just not yet assigned. The second line **throws** `ReferenceError: Cannot access 'b' before initialization` — `let/const` are also hoisted, but remain in the **Temporal Dead Zone** (1.5) until their declaration line actually executes, so accessing them beforehand is a hard error rather than a silent undefined.

hoisting declaration vs assignment

1.3

BASIC

CONCEPT

What is a closure?

A closure is a function bundled together with references to its surrounding (enclosing) lexical scope — it lets a function keep accessing variables from where it was *defined*, even after that outer scope has finished executing and would otherwise be gone.

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = makeCounter();  
counter(); // 1  
counter(); // 2 — "count" persisted between calls
```

`makeCounter()` has already returned, yet the inner function still has live access to `count` — it wasn't garbage collected because the returned function still references it. This is the exact mechanism behind private state in the module pattern, memoization (1.6), and debounce/throttle (7.4/7.5).

closure

lexical scope

1.4

INTERMEDIATE

DEBUG

The classic var-in-a-loop trap — trace it and fix it.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
  
// prints: 3, 3, 3 — not 0, 1, 2
```

var is function-scoped, so there is only *one* i shared across every loop iteration and every closure created inside it. By the time any of the three setTimeout callbacks actually runs (after the loop has fully finished, per Section 5's event-loop rules), i has already reached 3 — every callback closes over the exact same variable, and reads its final value.

```
// Fix 1: let — a fresh binding of i is created per iteration  
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
  
// prints: 0, 1, 2  
  
// Fix 2 (pre-ES6 / the historical answer): an IIFE captures i by value per iteration  
for (var i = 0; i < 3; i++) {  
  (function (captured) {  
    setTimeout(() => console.log(captured), 0);  
  })(i);  
}
```

per-iteration binding

IIFE

REAL INTERVIEW SIGNAL

Search sources call this out by name as the "classic closures inside a for-loop with var trap" — explicitly flagged as something to be ready to write and explain on a whiteboard, not just recognize.

1.5

INTERMEDIATE

CONCEPT

What is the Temporal Dead Zone?

The TDZ is the span of code between the start of a scope and the point where a let/const variable is actually declared, during which the variable exists (it's been hoisted) but cannot legally be accessed — any reference throws a ReferenceError rather than returning undefined. This is a deliberate design choice, not an implementation quirk: it converts a class of subtle "used before intended" bugs that var allows to pass silently (1.2) into loud, immediate errors instead.

Temporal Dead Zone

TDZ

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = (n) => { for (let i=0;i<1e8;i++); return n*n; };
const fastSquare = memoize(slowSquare);
fastSquare(5); // slow the first time
fastSquare(5); // instant - served from the closure's cache
```

The returned function is a closure over `cache` — private state that persists across calls but is entirely inaccessible from outside, since nothing else has a reference to that specific `Map`. `JSON.stringify` as a cache key is a simple, common choice, with a real limitation worth naming: it breaks for arguments that aren't reliably serializable (functions, undefined mixed with key ordering, circular references).

memoization

private state via closure

```
function createUserWidget(user) {
  return {
    getName: () => user.name,
    rename: (newName) => { user.name = newName; }
  };
}

const widget = createUserWidget({ name: "Alice" });
console.log(widget.getName()); // "Alice"
widget.rename("Bob");
console.log(widget.getName()); // "Bob" - this is correct, not stale
```

This particular case is actually *not* a bug — it's worth an interviewer presenting exactly this to see whether a candidate incorrectly assumes closures "snapshot" a value at creation time. A closure captures a **reference** to the variable/object, not a frozen copy — so both `getName` and `rename` share the exact same `user` object, and a mutation through one is visible through the other, correctly. The *real* stale-data bug shows up when a closure captures a **primitive** passed by value at creation time and the caller wrongly expects it to track later changes to some outer variable — or, far more commonly in practice, in exactly 1.4's shared-loop-variable pattern applied to real application state instead of a toy `setTimeout` example. The debugging instinct being tested: closures over objects share live references (mutations visible everywhere); closures over primitives captured by value do not (later reassignment of the outer variable is invisible to an already-created closure unless it re-reads a shared enclosing binding, as in the loop case).

reference capture, not value snapshot

Block scope vs function scope — and how IIFEs relate to the module pattern.

Function scope (the only kind `var` respects) means a variable is visible anywhere inside the enclosing function, regardless of nested `{}` blocks (`if`, `for`, bare braces). **Block scope** (`let/const`) confines a variable to the nearest enclosing `{}`. Before `let/const` existed, the only way to get a private, non-global scope at all was to wrap code in a function — and since a plain function declaration pollutes the enclosing scope with its own name, the **Immediately Invoked Function Expression** pattern `((function(){ ... }()))` became the standard idiom: define an anonymous function and call it immediately, creating a throwaway function scope purely to keep internal variables private. This is the direct historical ancestor of the module pattern (private state + a returned public API, structurally identical to 1.6's memoize closure) and, later, of real ES module scoping — understanding IIFEs is understanding *why* JS modules needed dedicated language support instead of just using blocks.

IIFE module pattern

WHAT THE WEB SOURCES EMPHASIZE

Multiple 2026 interview-prep sources independently state that frontend interviews "don't ask you to define closure — they ask you to use one, under a timer, live-shared," directly shaping this section's emphasis on the loop trap (1.4) and a real memoize implementation (1.6) over pure definitions.

02 this, Execution Context & Binding

Sources frame this cluster as directly predictive of real-world bugs: "the way this breaks is the way real bugs arrive."

2.1 BASIC CONCEPT How is this determined?

The single most important fact about `this` in ordinary (non-arrow) functions: it's determined entirely by **how the function is called** — the call site — not where or how it was defined.

- **Method call** (`obj.method()`) — `this` is `obj`.
- **Plain function call** (`fn()`) — `this` is undefined in strict mode (the module/class default), or the global object in sloppy mode.
- **Constructor call** (`new Fn()`) — `this` is the newly created object.
- **Explicit binding** (`call`/`apply`/`bind`, 2.2) — `this` is whatever is explicitly passed.
- **Arrow function** — has no `this` of its own at all; it lexically inherits `this` from its enclosing scope (2.4).

call site

implicit vs explicit binding

2.2 BASIC COMPARISON call vs apply vs bind.

	CALL	APPLY	BIND
Invokes immediately?	Yes	Yes	No — returns a new function
Arguments	Individually: <code>fn.call(ctx, a, b)</code>	As an array: <code>fn.apply(ctx, [a, b])</code>	Individually, optionally partial: <code>fn.bind(ctx, a)</code>

```
function greet(greeting) { return `${greeting}, ${this.name}`; }
const user = { name: "Alice" };

greet.call(user, "Hi");           // "Hi, Alice"
greet.apply(user, ["Hi"]);        // "Hi, Alice"
const bound = greet.bind(user);
bound("Hi");                     // "Hi, Alice" — callable later, this stays fixed
```

call

apply

bind

2.3

BASIC

OUTPUT

Regular method vs arrow function property — what prints?

```
const obj = {
  name: "Widget",
  regular: function () { return this.name; },
  arrow: () => { return this.name; }
};

console.log(obj.regular()); // "Widget"
console.log(obj.arrow());   // undefined
```

regular is called as `obj.regular()` — a method call, so `this` is `obj` (2.1's first rule), and `this.name` is `"Widget"`. `arrow` is an arrow function, which has no `this` of its own — it lexically captures `this` from the scope *surrounding the object literal* (2.4), typically the module or global scope, where `name` doesn't exist — hence `undefined`, not an error, since accessing a missing property on an existing object just returns `undefined`.

method call this

lexical this

2.4

INTERMEDIATE

CONCEPT

Why don't arrow functions have their own `this`, and when does that matter?

Arrow functions were deliberately designed without their own `this` binding, arguments object, or prototype — they inherit `this` lexically, exactly like an ordinary variable reference, from whatever scope textually encloses them at definition time. Critically, **call, apply, and bind have no effect on an arrow function's `this`** — there's no dynamic binding to override.

This matters most in two recurring real situations: (1) **as a class field/method** — `onClick = () => { this.setState(...) }` as a class field is popular specifically because it permanently locks `this` to the instance, immune to how the callback is later invoked (a direct fix for 2.6's bug); (2) **as a top-level object method** (2.3's exact trap) — using an arrow function for a method that's supposed to reference the containing object is a common, genuine bug, not a stylistic choice, since the object literal itself creates no new lexical scope for `this` to bind to.

lexical this

arrow functions ignore call/apply/bind

```
Function.prototype.myBind = function (context, ...boundArgs) {  
  const originalFn = this; // the function myBind was called on  
  return function (...callArgs) {  
    return originalFn.apply(context, [...boundArgs, ...callArgs]);  
  };  
};  
  
function greet(greeting, punctuation) {  
  return `${greeting}, ${this.name}${punctuation}`;  
}  
const bound = greet.myBind({ name: "Alice" }, "Hi");  
bound("!"); // "Hi, Alice!"
```

The key insight the polyfill tests: `this` inside `myBind` itself is the function it was called on (`greet`), captured in `originalFn` via closure (1.3) — the returned function is itself a closure over both `originalFn` and `context`, and uses `apply` (2.2) internally to actually invoke the original function with the fixed context, merging any arguments pre-supplied at bind time with any supplied at call time.

bind polyfill

closure + apply

A class method passed as a callback loses this. Diagnose and fix it three ways.

```
class Button {
  constructor(label) { this.label = label; }
  handleClick() { console.log(`Clicked: ${this.label}`); }
}

const btn = new Button("Submit");
element.addEventListener("click", btn.handleClick);
// TypeError: Cannot read properties of undefined (reading 'label')
```

Passing `btn.handleClick` as a bare reference detaches the method from `btn` — per 2.1, this is decided entirely by the call site, and the event listener calls it as a plain function call (effectively `handleClick()`), not as `btn.handleClick()`, so `this` is undefined (strict mode, the default inside a class body).

```
// Fix 1: bind explicitly at the call site
element.addEventListener("click", btn.handleClick.bind(btn));

// Fix 2: wrap in an arrow function, which captures the surrounding this
element.addEventListener("click", () => btn.handleClick());

// Fix 3: declare the method as an arrow function class field (2.4)
class Button {
  constructor(label) { this.label = label; }
  handleClick = () => { console.log(`Clicked: ${this.label}`); }
}
element.addEventListener("click", btn.handleClick); // now safe
```

detached method call

three binding fixes

WHAT INTERVIEWERS LISTEN FOR

Naming all three fixes — and explaining the trade-off (class-field arrows create a new function per instance; bind at the call site is explicit but easy to forget) — is what separates a memorized fix from real understanding.

```
"use strict";
function whoAmI() { return this; }
console.log(whoAmI()); // undefined

// without "use strict" (sloppy mode):
function whoAmI2() { return this; }
console.log(whoAmI2()); // the global object (window in browsers, globalThis)
```

This is a real historical wart 2.1 glosses over: in non-strict ("sloppy") mode, a plain function call that would otherwise have `this === undefined` instead silently defaults `this` to the global object — a legacy behavior preserved for backward compatibility, and a genuine source of accidental global-object mutation bugs in older code. ES modules and class bodies are *always* strict mode implicitly, which is exactly why this specific trap has become rarer in modern codebases — it's now something you mostly encounter reading, or being asked about, legacy sloppy-mode scripts rather than writing new code.

`strict mode this``sloppy mode default-to-global`

WHAT THE WEB SOURCES EMPHASIZE

Search sources are explicit that `call`/`apply`/`bind` and arrow-function `this` behavior are treated as one connected cluster in real interviews, not separate topics — directly justifying why 2.4–2.6 build on each other rather than being independent questions.

03 Objects & Prototypal Inheritance

MDN and GeeksforGeeks converge on the same framing: ES6 `class` is syntactic sugar over the prototype chain, and interviewers use this section to confirm you know what's underneath the sugar.

3.1 BASIC CONCEPT What is the prototype chain?

Every JS object has an internal link to another object, its **prototype**, which serves as a fallback source for properties/methods it doesn't have directly. Property lookup (`obj.foo`) checks the object itself first; if not found, it walks up to the object's prototype, then that prototype's prototype, and so on, until either the property is found or the chain ends at `null` (the top of every chain — `Object.prototype`'s own prototype). This is how every plain object inherits methods like `.toString()` or `.hasOwnProperty()` without ever defining them directly — they live on `Object.prototype`, at the very end of the chain.

prototype chain

property lookup fallback

3.2 BASIC COMPARISON `Object.create` vs `new` vs `class`.

	<code>Object.create(proto)</code>	<code>new fn()</code>	<code>class</code>
What it does	Creates an object with the given prototype directly, no constructor function needed	Creates a new object, links its prototype to <code>fn.prototype</code> , runs <code>fn</code> with <code>this</code> bound to it	Declarative syntax generating the same new-based mechanism (3.4)
Explicitness	Most explicit/low-level	Function-based, pre-ES6 idiom	Most readable, most common in modern code

All three ultimately produce the same underlying structure — an object with a prototype link — `Object.create` is simply the most direct way to state that intent, while `new` and `class` add constructor-function conveniences on top.

`Object.create`

constructor function

3.3

BASIC

OUTPUT

__proto__ vs prototype — what's the difference?

```
function Dog(name) { this.name = name; }
Dog.prototype.bark = function () { return `${this.name} says woof`; };

const rex = new Dog("Rex");

console.log(rex.prototype);      // undefined — instances don't have .prototype
console.log(Dog.prototype);     // { bark: [Function] } — the constructor's prototype object
console.log(rex.__proto__ === Dog.prototype); // true
```

`.prototype` is a property that exists only on **functions** (specifically, ones meant to be used as constructors) — it's the object that will become the prototype of every instance created with `new`.

`.__proto__` is a property (a legacy accessor, now more properly accessed via `Object.getPrototypeOf()`) that exists on **instances**, pointing to whatever object actually *is* their prototype. Confusing "the object that will be assigned as a prototype" (`Fn.prototype`) with "the object's actual current prototype" (`instance.__proto__`) is one of the most common sources of prototype confusion in interviews.

`Fn.prototype``instance.__proto__`

3.4 INTERMEDIATE COMPARISON ES6 classes — what are they sugar over?

```
// Class syntax
class Dog {
  constructor(name) { this.name = name; }
  bark() { return `${this.name} says woof`; }
}

// What it compiles down to, conceptually
function Dog(name) { this.name = name; }
Dog.prototype.bark = function () { return `${this.name} says woof`; };
```

Under the hood, every method defined in a class body — other than the constructor — is attached to `ClassName.prototype`, exactly as with the manual function-plus-prototype pattern. This is why an instance method is shared, in memory, across every instance (looked up via the prototype chain, 3.1) rather than duplicated per object — a real, meaningful memory/performance detail, not just syntax trivia. What `class` syntax adds beyond pure sugar: methods defined this way are non-enumerable by default (unlike manually assigned prototype properties), and calling a class constructor without `new` throws an error instead of silently misbehaving.

class = sugar over prototype + constructor function

3.5 INTERMEDIATE COMPARISON Object.freeze vs Object.seal.

	OBJECT.FREEZE	OBJECT.SEAL
Add new properties?	No	No
Delete properties?	No	No
Modify existing property values?	No	Yes

`freeze` is the strictest — a genuinely immutable object (for its own direct properties; nested objects are *not* automatically frozen, a common gotcha — freezing is shallow). `seal` locks the object's shape (no adding/removing keys) but still allows updating existing values. Both are the standard, direct mitigation for **prototype pollution** (Section 9) when applied specifically to `Object.prototype`: `Object.freeze(Object.prototype)` makes it impossible for an attacker-controlled merge to add malicious properties there.

Object.freeze Object.seal shallow freeze

```
function double(n) { n = n * 2; return n; }
let x = 5;
double(x);
console.log(x); // 5 – primitives are passed by value

function addItem(arr) { arr.push("new"); }
let list = ["a", "b"];
addItem(list);
console.log(list); // ["a", "b", "new"] – objects/arrays are passed by reference
```

Primitives (numbers, strings, booleans, null, undefined, symbols, bigints) are copied by value on assignment or when passed as arguments — the function receives its own independent copy, and reassigning the parameter inside the function has zero effect on the caller's variable. Objects and arrays are passed by **reference** — the function receives a copy of the reference (the "pointer"), which still points at the exact same underlying object, so mutating it (`.push`, setting a property) is visible to the caller, while *reassigning* the parameter itself (`arr = []`) would not be, since that only rebinds the local reference variable, not the original one.

pass by value

pass by reference (of the pointer)

```
function merge(target, source) {
  for (const key in source) {
    if (typeof source[key] === "object") {
      target[key] = target[key] || {};
      merge(target[key], source[key]); // no key filtering - vulnerable
    } else {
      target[key] = source[key];
    }
  }
  return target;
}

const malicious = JSON.parse('{"__proto__": {"isAdmin": true}}');
merge({}, malicious);

const innocentObject = {};
console.log(innocentObject.isAdmin); // true - every object is now "admin"!
```

Prototype pollution occurs when attacker-controlled input reaches an unsafe recursive merge/clone/assignment operation that doesn't filter dangerous keys like `__proto__`, `constructor`, or `prototype` — allowing the attacker to write directly onto **Object.prototype** itself, which every plain object in the entire application inherits from (3.1). The result isn't limited to one object; it silently corrupts the shared prototype every object in the runtime relies on, and per search sources, on the client this frequently escalates into **DOM-based XSS** (9.1) if any polluted property later gets rendered into the page.

prototype pollution

`__proto__` key injection

Mutating a shared prototype method accidentally affects every instance. Diagnose it.

```

class Widget {
  constructor(name) { this.name = name; }
  render() { return `
${this.name}
`; }
}

const a = new Widget("A");
const b = new Widget("B");

a.render = function () { return `${this.name}`; }; // intended: only affect "a"

console.log(a.render()); // "A"
console.log(b.render()); // "
B
" — unaffected, this part is correct

// The actual bug shows up here instead:
Widget.prototype.render = function () { return `${this.name}`; };
console.log(b.render()); // "B" — every instance, past and future, changes at once

```

Assigning directly to an *instance* (`a.render = ...`) is safe and local — per 3.1's lookup rule, it shadows the prototype method for that one instance only, since the instance's own property is found before the chain is even walked. The real bug is assigning to **Widget.prototype.render** directly — every existing and future instance shares that exact same prototype object (3.4), so the change is instantly visible everywhere, often unintentionally, especially when the mutation happens deep inside a library or a "helper" function the original author didn't realize was touching a shared prototype rather than a single instance.

instance property shadows prototype

shared prototype mutation

WHAT THE WEB SOURCES EMPHASIZE

MDN and GeeksforGeeks both explicitly frame ES6 classes as "syntactic sugar" over the prototype mechanism, and search sources on prototype pollution specifically link it forward to DOM-based XSS — both direct citations used verbatim in 3.4 and 3.7 rather than paraphrased loosely.

The section built almost entirely from output-prediction snippets — precisely the format search sources say 2026 interviews favor over pure definitions.

4.1 BASIC COMPARISON **== vs === — is == ever acceptable?**

=== (strict equality) compares value *and* type, with no conversion — `1 === "1"` is false. == (loose equality) first coerces operands to a common type, then compares — `1 == "1"` is true, following a specific, non-obvious set of coercion rules that are a common source of bugs (4.2). The standard, defensible production guidance: default to === everywhere, since it's predictable and doesn't require memorizing coercion rules to reason about. The one narrow, genuinely common exception: `x == null` is a widely-used, deliberate idiom that checks for *both* null and undefined in a single comparison (since `null == undefined` is specifically true, while `null == 0` or `null == false` are both false) — a rare case where the loose-equality coercion rule is precise and useful rather than a footgun.

strict vs loose equality

`x == null` idiom

4.2 BASIC OUTPUT **Classic coercion snippets — what do these print?**

```
console.log([] + []);      // ""
console.log([] + {});      // "[object Object]"
console.log('5' + 3);      // "53"
console.log('5' - 3);      // 2
console.log(1 + true);     // 2
```

The + operator has two personalities: if *either* operand is (or coerces to) a string, it performs string concatenation; otherwise, numeric addition. Arrays and objects are first coerced to primitives via their own `toString()` — an empty array becomes "" (hence `[] + [] === ""`), a plain object becomes "[object Object]". Because `"5" + 3` sees a string on the left, it concatenates: "53". **The - operator has no string-concatenation meaning at all** — it always coerces both operands to numbers, so `'5' - 3` is 2, not a type error and not a string — this operator asymmetry (+ can mean two different things, - only ever means one) is exactly what most output-prediction "gotcha" questions are testing.

+ operator ambiguity

ToPrimitive coercion

4.3 BASIC GOTCHA **typeof gotchas.**

```
console.log(typeof null);           // "object" ← famous historical bug
console.log(typeof NaN);           // "number" ← NaN is a number value
console.log(typeof undeclaredVar); // "undefined" (no error, unlike referencing it directly)
console.log(typeof function(){});  // "function"
console.log(typeof []);            // "object" ← use Array.isArray() instead
```

`typeof null === "object"` is a widely-known bug baked into JS's very first implementation and preserved ever since for backward compatibility — it can never be fixed without breaking the web. `NaN` genuinely is of type `"number"` (4.4) — it represents an invalid number, not an absence of one. `typeof` on a never-declared identifier is one of the only ways to safely probe for a variable's existence without triggering a `ReferenceError`.

`typeof null bug` `Array.isArray`

4.4 INTERMEDIATE CONCEPT **Why does `NaN === NaN` return false, and how do you check for it?**

```
console.log(NaN === NaN);           // false
console.log(Number.isNaN(NaN));     // true – correct check
console.log(isNaN("hello"));       // true – but for the WRONG reason (coerces first)
console.log(Number.isNaN("hello")); // false – correct: "hello" is not the NaN value, just not a r
```

Per the IEEE 754 floating-point standard JS numbers follow, `NaN` is specifically defined as never equal to anything, including itself — this is standards-mandated behavior, not a JS-specific quirk. The global `isNaN()` function is a common trap: it coerces its argument to a number *before* checking, so `isNaN("hello")` is true not because `"hello"` is the `NaN` value, but because converting it to a number produces `NaN`. `Number.isNaN()` (ES6) is the precise version — it returns true only for the actual `NaN` value, with no coercion, and is the version that should be used in new code.

`NaN never equals itself` `Number.isNaN vs isNaN`

4.5

INTERMEDIATE

GOTCHA

Truthy vs falsy — the full list, and the gotcha.

JavaScript has exactly **eight falsy values**: `false`, `0`, `-0`, `0n` (BigInt zero), `""` (empty string), `null`, `undefined`, and `NaN`. Everything else is truthy.

```
if ([]) console.log("truthy"); // prints "truthy" – an empty array IS truthy
if ({}) console.log("truthy"); // prints "truthy" – an empty object IS truthy too
```

The gotcha this consistently exposes: an empty array or empty object is **not** in the falsy list — they're both objects, and every object is truthy regardless of how "empty" it looks, which trips up developers who reason "empty means falsy" by analogy with the empty string. Checking whether an array/object is actually empty requires an explicit check (`arr.length === 0`, `Object.keys(obj).length === 0`), never a bare truthiness test.

8 falsy values

empty array/object are truthy

4.6

ADVANCED

OUTPUT

Full trace through a mixed coercion chain.

```
let x = "0";
if (x) {
  console.log("A");
} else {
  console.log("B");
}
// "A" – a non-empty string is truthy, even the string "0"

console.log("0" == 0); // true – string coerced to number: 0 == 0
console.log("0" == false); // true – false coerced to 0, then "0" coerced to 0
console.log("" == false); // true – both sides coerce to 0
console.log(null == 0); // false – null only loosely equals undefined, by spec special-case (4.2.1)
```

The core trap this walks through: `"0"` is truthy in a boolean context (4.5's rule — any non-empty string is truthy, full stop, no numeric interpretation happens) but `"0" == false` is separately true under `==`'s coercion rules — two entirely different algorithms (ToBoolean for truthiness, the multi-step `==` abstract equality algorithm for loose comparison) that happen to disagree on this exact value, which is precisely why treating "truthy/falsy" and "loosely equal" as the same mental model is a reliable source of bugs.

ToBoolean ≠ == coercion

```
console.log(NaN === NaN);           // false
console.log(Object.is(NaN, NaN)); // true

console.log(0 === -0);              // true
console.log(Object.is(0, -0));      // false
```

`Object.is()` behaves almost identically to `===`, with exactly two deliberate differences: it correctly treats `NaN` as equal to itself (fixing 4.4's surprise), and it correctly distinguishes `+0` from `-0` as genuinely different values (which `===` treats as equal, even though they're distinguishable — `1 / 0` is `Infinity` but `1 / -0` is `-Infinity`). `Object.is` is used internally by React's reconciliation for exactly this reason — it needs a comparison that's precise about these specific edge cases when deciding whether a value has genuinely changed.

`Object.is``+0 vs -0`

WHAT THE WEB SOURCES EMPHASIZE

Multiple 2026 prep guides explicitly list type coercion alongside the event loop and closures as one of the handful of topics separating strong from weak candidates — and consistently present it via short output-prediction snippets rather than abstract rule recitation, directly shaping this section's format.

05 Async JS: Event Loop, Promises & Async/Await

Grounded directly in MDN's own microtask-guide language. This is the section where a candidate's ability to trace, not just describe, execution order is actually tested.

5.1 BASIC CONCEPT The call stack, and why JS is "single-threaded."

The **call stack** tracks currently executing function calls — each call pushes a new frame, each return pops one off. JavaScript has exactly one call stack (one thread of execution for JS code itself), meaning only one piece of JS can run at any instant — there's no true parallel execution of JS statements the way multi-threaded languages allow. This is precisely why a long-running synchronous computation **blocks everything** — no event handler, no timer callback, no promise resolution can run until the stack is empty again, since there's simply no other thread available to run them concurrently.

call stack

single-threaded

5.2 BASIC CONCEPT What is the event loop?

Since JS itself is single-threaded (5.1), asynchronous work (timers, network requests, I/O) is actually handled by the surrounding runtime environment (the browser, or libuv in Node — Section 6), not by JS itself. The **event loop** is the mechanism that coordinates handing results of that external work back into JS's single call stack: it continuously checks whether the call stack is empty, and if so, pulls the next queued piece of work — first fully draining the microtask queue, then taking one task from the macrotask queue — and pushes it onto the stack to run. The mental model that matters most for interviews: *synchronous code always finishes completely before any queued asynchronous callback gets a chance to run*, no matter how that callback was scheduled.

event loop

runtime environment handles async work

5.3 BASIC COMPARISON Microtask queue vs macrotask (task) queue.

	MICROTASK QUEUE	MACROTASK (TASK) QUEUE
Contains	Promise <code>.then/.catch/.finally</code> , <code>queueMicrotask</code> , <code>MutationObserver</code>	<code>setTimeout</code> , <code>setInterval</code> , I/O, UI events
Drain behavior	ALL microtasks run before the next macrotask — including new ones added while draining	Exactly ONE task runs per event-loop iteration
Priority	Higher — always fully drained first	Lower

Per MDN directly: after every single macrotask finishes, the engine runs *every* microtask currently queued — including ones scheduled by other microtasks in the process — before it's allowed to move on to rendering or the next macrotask. This "drain completely, including newly added ones" behavior is exactly what makes 5.4's ordering puzzle deterministic rather than ambiguous.

microtask queue

macrotask/task queue

5.4

INTERMEDIATE

OUTPUT

The classic ordering puzzle — trace it fully.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

```
// Output: 1, 4, 3, 2
```

1. `console.log("1")` runs immediately (synchronous) → 1
2. `setTimeout(...)` schedules its callback on the macrotask queue and returns immediately — it does NOT run yet, even with a 0ms delay.
3. `Promise.resolve().then(...)` schedules its callback on the microtask queue and returns immediately.
4. `console.log("4")` runs immediately (still synchronous) → 4
5. The main synchronous script has now finished — the call stack is empty. Per 5.3, the event loop *fully drains the microtask queue first*: the `.then` callback runs → 3
6. Only now does the event loop move to the macrotask queue and run the `setTimeout` callback → 2

The single most important, most commonly missed rule: `setTimeout(fn, 0)` does *not* mean "run immediately" — it means "run as soon as possible after the current synchronous code AND all pending microtasks finish," which is always strictly later than any already-queued microtask, regardless of the requested delay.

`setTimeout 0` is never truly 0

microtasks before macrotasks

REAL INTERVIEW SIGNAL

This exact ordering pattern — mixing synchronous logs, a Promise, and `setTimeout` — is specifically cited across search sources as the classic senior-level event-loop question, used precisely because getting the order right requires real understanding, not memorization.

5.5

INTERMEDIATE

CONCEPT

What is a Promise, and its three states?

A Promise is an object representing the eventual result of an asynchronous operation, in exactly one of three states: **pending** (not yet resolved or rejected), **fulfilled** (completed successfully, holding a value), or **rejected** (failed, holding a reason/error). A Promise transitions state exactly once — from pending to either fulfilled or rejected, permanently — which is precisely what makes `.then()` callbacks reliable: they're guaranteed to fire exactly once for a given settled state, never re-fire, never fire twice.

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    success ? resolve("done") : reject(new Error("failed"));
  }, 1000);
});

promise
  .then(value => console.log(value))    // runs if fulfilled
  .catch(err => console.error(err));    // runs if rejected
```

pending/fulfilled/rejected

settles exactly once

```
// async/await
async function getUser(id) {
  const response = await fetch(`/users/${id}`);
  const data = await response.json();
  return data;
}

// equivalent Promise chain
function getUser(id) {
  return fetch(`/users/${id}`)
    .then(response => response.json())
    .then(data => data);
}
```

async/await is syntactic sugar over Promises, specifically designed to let asynchronous code *read* like synchronous code without actually blocking the call stack (5.1). An async function always returns a Promise, wrapping its return value automatically. `await` pauses execution of *just that async function* at that exact line until the awaited Promise settles — critically, it does not pause the entire program: other code (other event-loop turns, other async functions) continues running normally while this one function is suspended, exactly as if the rest of the function body were the body of a `.then()` callback.

async function returns a Promise

await pauses only that function

```
async function loadData() {  
  const data = await fetch("/api/data").then(r => r.json());  
  return data;  
}  
  
loadData(); // if this rejects, the error is silently swallowed –  
            // "Uncaught (in promise)" in the console, no app-level handling
```

An async function that throws (or awaits a rejected Promise) doesn't throw synchronously to its caller — it returns a *rejected Promise*, per 5.6. Calling `loadData()` without either `await`-ing it inside another `try/catch`, or attaching a `.catch()` directly, means that rejection has nowhere to go — it becomes an **unhandled promise rejection**, silently logged by the runtime rather than caught by any application code.

```
// Fix: wrap the await in try/catch inside the function...  
async function loadData() {  
  try {  
    const data = await fetch("/api/data").then(r => r.json());  
    return data;  
  } catch (err) {  
    console.error("Failed to load data:", err);  
    throw err; // or handle it here directly  
  }  
}  
  
// ...AND/OR handle it at the call site, since try/catch only catches what's inside it  
loadData().catch(err => console.error("Caller-level handling:", err));
```

The debugging instinct being tested: a `try/catch` around an `await` only catches errors from that specific awaited expression within that function — it does nothing for the *caller's* handling of the returned promise, which is exactly why production code often needs both layers, not just one.

unhandled promise rejection

try/catch scope vs caller `.catch`

```
console.log("start");

async function main() {
  console.log("A");
  await null; // a microtask suspension point, even for an already-resolved value
  console.log("B");
  setTimeout(() => console.log("C"), 0);
  await null;
  console.log("D");
}

main();
console.log("end");
// Output: start, A, end, B, D, C
```

1. "start" logs synchronously.
2. `main()` is called; it runs synchronously up to the first `await` — "A" logs.
3. `await null` immediately suspends `main`, scheduling its continuation as a **microtask** — even though `null` isn't a "real" pending Promise, `await` always yields control at least once, deferring the rest of the function.
4. Control returns to the caller; "end" logs synchronously (the main script continues, since `main()`'s internal suspension doesn't block anything outside it, per 5.6).
5. The main synchronous script finishes → microtask queue drains → `main`'s continuation runs: "B" logs, `setTimeout` schedules "C" as a macrotask, then the second `await null` suspends again, queuing another microtask.
6. That microtask is still in the same drain cycle (5.3's "keep draining newly added ones" rule) → runs immediately: "D" logs.
7. Microtask queue is now empty; the event loop finally moves to the macrotask queue → "C" logs.

await always yields at least one microtask turn

Promise.all vs allSettled vs race vs any.

METHOD	SETTLES WHEN	RESULT
Promise.all	All fulfill, OR any one rejects	Array of values, or rejects with the first error (others' results are lost)
Promise.allSettled	Every promise has settled (fulfilled or rejected)	Array of {status, value/reason} for every one — never itself rejects
Promise.race	The first promise to settle (fulfill OR reject)	That single result — winner takes all, others are ignored
Promise.any	The first promise to <i>fulfill</i>	That value; rejects only if <i>every</i> promise rejects

Use `all` when every result is required and any single failure should abort the whole operation. Use `allSettled` when you want every result regardless of individual failures (e.g. firing several independent API calls and reporting on each outcome separately). Use `race` for a timeout pattern (racing a real request against a rejecting timer). Use `any` when you have several redundant sources and only need the first successful one, tolerating individual failures among the rest.

`Promise.all``allSettled``race``any`

WHAT THE WEB SOURCES EMPHASIZE

MDN's own microtask guide was used directly for the drain-order rule underlying 5.3/5.4/5.8, and multiple 2026 sources independently single out a synchronous-log/Promise/setTimeout mix as the specific "senior-level" question that separates candidates — directly justifying the depth given to 5.4 and 5.8's full traces.

06 Node.js Runtime, Modules & Streams

Checked directly against the Node.js Learn docs. This is where "the event loop" from Section 5 gets a specific, non-browser implementation with its own rules.

6.1 BASIC COMPARISON Node's event loop vs the browser's.

Both run on the same fundamental idea (5.2) — one call stack, an event loop coordinating async work — but Node's is implemented by **libuv**, a C library providing an explicitly **phased** loop (6.3) rather than the browser's simpler task/microtask model, because Node needs to handle OS-level concerns browsers don't: file I/O, TCP sockets, child processes, and timers all need their own dedicated points in the loop. Node also adds its own extra, even-higher-priority microtask-like queue, `process.nextTick()` (6.4), which has no browser equivalent at all.

libuv phased event loop

6.2 BASIC COMPARISON CommonJS vs ES Modules.

	COMMONJS	ES MODULES (ESM)
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous-friendly (supports top-level <code>await</code>)
Resolution	Dynamic — any expression can be required	Static — imports are analyzable/hoisted at parse time, enabling tree-shaking
Caching	<code>require</code> is cached — a module is a singleton (6.8)	Also cached, per module specifier
Node-specific globals	<code>__dirname</code> , <code>__filename</code> available	Not available — use <code>import.meta.url</code> instead

ESM's *static* import structure — every import must be a literal, top-level string, resolvable without running any code — is precisely what makes tree-shaking (removing unused exports at build time) possible; CommonJS's fully dynamic `require()` (which can be called conditionally, inside a function, with a computed string) cannot be statically analyzed the same way.

CommonJS ESM static vs dynamic imports

timers	← setTimeout / setInterval callbacks
pending callbacks	← deferred I/O callbacks (e.g. some TCP errors)
idle, prepare	← internal libuv bookkeeping only
poll	← retrieve new I/O events; most app code runs here
check	← setImmediate() callbacks
close callbacks	← e.g. socket.on('close', ...)
(loop back to timers)	

Per Node's own docs, the **poll** phase is where most application logic actually executes (finished I/O callbacks). Microtasks (Promises, `queueMicrotask`) and `process.nextTick()` (6.4) aren't a phase of this diagram at all — they're drained entirely, between *every* phase transition and between every individual callback, exactly mirroring 5.3's "always drain first" rule from the browser model, just applied more granularly throughout all six phases instead of once per task.

timers/poll/check phases

nextTick vs Promise microtasks vs setImmediate vs setTimeout(fn, 0).

MECHANISM	QUEUE	PRIORITY
<code>process.nextTick()</code>	Node's own dedicated queue	Highest — drained before Promise microtasks, and before moving to the next phase
<code>Promise .then</code>	Standard microtask queue	Second — after all pending nextTick callbacks
<code>setImmediate()</code>	Check phase	Runs after the poll phase completes, in the check phase specifically
<code>setTimeout(fn, 0)</code>	Timers phase	Runs in the timers phase — ordering relative to <code>setImmediate</code> from the main module is technically unspecified, though inside an I/O callback, <code>setImmediate</code> reliably fires before a 0ms timeout

`process.nextTick` priority`setImmediate` vs `setTimeout(0)`6.5 INTERMEDIATE CONCEPT **What is backpressure, and how does `.pipe()` handle it?**

Node streams process data in chunks instead of loading an entire file/payload into memory at once — essential for large files or network payloads. **Backpressure** is what happens when a writable destination can't consume data as fast as a readable source produces it — without any mechanism to slow the source down, data piles up in memory buffers indefinitely, a real, documented way to exhaust memory on a production server processing large streams.

```
readableStream.pipe(writableStream);  
// .pipe() automatically pauses the readable source when the writable's  
// internal buffer is full (above its highWaterMark), and resumes it  
// once the writable signals it's ready for more – no manual buffering needed
```

`.pipe()` handles this automatically: it monitors the writable stream's return value from `.write()` (which signals whether its internal buffer is under or over its configured `highWaterMark`), pausing the readable source when the writable is overwhelmed and resuming it once drained — the same coordination a hand-rolled version would need to implement manually via `.pause()/resume()` and the `'drain'` event.

`backpressure``highWaterMark``pipe()` auto-handles flow

```
console.log("start");

setTimeout(() => console.log("setTimeout"), 0);
setImmediate(() => console.log("setImmediate"));
process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("promise"));

console.log("end");

// Output: start, end, nextTick, promise, setTimeout, setImmediate
// (setTimeout/setImmediate order can flip depending on process startup
// overhead when run from the top-level main module – this is
// documented, expected non-determinism, not a bug to "fix")
```

Trace: synchronous code runs fully first ("start", "end"). The call stack empties → Node drains its two-tier microtask system in strict order, `process.nextTick`'s queue completely before the Promise microtask queue (6.4) → "nextTick", then "promise". Only then does the event loop proceed into its phases — `setTimeout`'s callback runs in the timers phase, `setImmediate`'s in the check phase (6.3), later in the same iteration. The parenthetical non-determinism between the last two is itself a fact worth stating in an interview — claiming perfect certainty about `setTimeout` vs `setImmediate` ordering from the top-level module is a documented over-claim.

nextTick before Promise, always

The event loop is blocked. Diagnose a synchronous CPU-bound bottleneck.

Because Node runs JS on a single thread (5.1, unchanged from the browser model), a long synchronous computation — a huge in-memory sort, an unoptimized regex on a large string, synchronous JSON parsing of a massive payload — occupies the call stack completely, and per 5.2's rule, *nothing else can run* until it finishes: no other request is served, no timer fires, no I/O callback executes, even on an otherwise idle, high-capacity server. Symptoms in production: request latency spikes uniformly across *all* concurrent requests at once (not just the slow one), and health-check/heartbeat timers start firing late or get dropped, since even those depend on the same blocked event loop.

Diagnosis: Node's built-in `--prof` flag or the `--inspect` flag with Chrome DevTools' CPU profiler attached will show a single, long-running synchronous call dominating the flame graph — the signature look is one wide, unbroken block rather than many short ones. The fix is architectural, not a quick patch: move genuinely CPU-bound work off the main thread entirely, via a `worker_thread`, a child process, or (for embarrassingly parallel batch work) an external job queue — the event loop itself cannot be made to "share time" with a single synchronous call; the call has to be moved elsewhere.

CPU-bound blocks the whole loop

`worker_threads`

6.8 **ADVANCED** GOTCHA **require() caching — why, and what production bug can it cause?**

Node caches every module the first time it's `require()`d, keyed by its resolved file path — every subsequent `require()` of that same path returns the exact same, already-executed module object, not a fresh re-execution. This is deliberate and usually desirable: it avoids re-running module-level setup code repeatedly and lets multiple files share one instance of, say, a database connection pool. The real production gotcha: because the cached module *is* a shared singleton, any mutable state a module holds (an in-memory cache, a counter, a config object with a mutable property) is **shared across every part of the application that requires it** — a bug in one seemingly unrelated part of the codebase mutating that shared object can silently affect completely different code elsewhere that also required the same module, with no explicit coupling visible in either file. This is a direct structural echo of 3.8's shared-prototype-mutation bug, at the module-system level instead of the object-prototype level — the same category of mistake (unexpectedly shared mutable state) recurring at a different layer of the language.

require cache = singleton

shared mutable module state

WHAT THE WEB SOURCES EMPHASIZE

Node's own "Node.js Learn" documentation was used directly for the phase diagram and nextTick-priority claims in 6.3/6.4, including the documented Node 20+/libuv 1.45 change to timer-phase ordering — current, version-specific detail rather than generic, possibly-outdated "the event loop has phases" folklore.

07 DOM, Events & Practical Coding Patterns

The live-coding section — Glassdoor reports converge specifically on debounce/throttle and DOM-tree-comparison tasks as real, asked coding exercises.

7.1 BASIC CONCEPT Event bubbling vs capturing.

When an event fires on a DOM element, it travels through the DOM tree in two possible phases: **capturing** (top-down — from the document root inward to the target element) happens first, then **bubbling** (bottom-up — from the target element back out to the root) happens after. By default, `addEventListener` attaches a listener to the bubbling phase; passing `{ capture: true }` attaches it to the capturing phase instead. For a click on a deeply nested button inside several `div`s, the event first travels *down* through every ancestor (capturing listeners fire, if any), reaches the button (the target), then travels back *up* through the same ancestors in reverse (bubbling listeners fire) — this bubble-up path is exactly what makes 7.2's event delegation pattern possible at all.

capturing phase

bubbling phase

7.2 BASIC CONCEPT What is event delegation?

Event delegation attaches a single listener to a common *ancestor* element rather than attaching a separate listener to every individual child — relying on 7.1's bubbling behavior to catch events from any child, current or future. Instead of "did this specific button get clicked," the ancestor's handler checks `event.target` to determine which specific child actually triggered the event. This is more efficient in two concrete ways: far fewer listener objects need to be created and held in memory for a list with many items, and — the more important production reason — it automatically handles elements added *after* the listener was attached (dynamically inserted list items, for instance), with zero extra code, since there's no need to re-attach a listener to every newly created child individually.

event delegation

`event.target`

```
// HTML:


- Item 1
- ...



document.getElementById("list").addEventListener("click", (event) => {
  const item = event.target.closest("li");
  if (!item) return; // click landed outside any <li> (e.g. on the <ul> padding)

  console.log("Clicked item:", item.dataset.id);
});

// New items added later work automatically — no re-attaching needed:
document.getElementById("list").insertAdjacentHTML(
  "beforeend", `


- Item 99


`
);
```

`.closest()` is the key tool here — it walks *up* from the actual click target to find the nearest matching ancestor, which correctly handles clicks on nested elements inside an `<li>` (an icon or a nested `<span>`), not just direct hits on the `<li>` itself.

```
event.target.closest()
```

```
function debounce(fn, delay) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}

const search = debounce((query) => console.log("Searching:", query), 300);
input.addEventListener("input", (e) => search(e.target.value));
// Only fires 300ms after the user STOPS typing – every keystroke resets the timer
```

Debounce guarantees a function runs only once the triggering event has stopped firing for the configured delay — every new call cancels and restarts the pending timer (1.3's closure holding `timeoutId` is what makes this possible). The classic interview follow-up, per search sources: this is a **trailing-edge-only** implementation (it fires after the pause). Production debounce utilities typically also support a **leading-edge** option (fire immediately on the first call, then suppress until the pause) — naming this gap unprompted is a strong signal.

debounce

trailing vs leading edge

```
function throttle(fn, limit) {
  let inCooldown = false;
  return function (...args) {
    if (inCooldown) return;
    fn.apply(this, args);
    inCooldown = true;
    setTimeout(() => { inCooldown = false; }, limit);
  };
}

const onScroll = throttle(() => console.log("scroll position:", window.scrollY), 200);
window.addEventListener("scroll", onScroll);
// fires AT MOST once every 200ms, throughout continuous scrolling
```

	DEBOUNCE	THROTTLE
Guarantee	Fires once, after the event stops	Fires at most once per fixed interval, continuously
Use case	Search-as-you-type (wait for the pause)	Scroll/resize handlers (need steady, periodic updates)

throttle

fires at most once per interval

Implement deep clone, and explain why the JSON shortcut is broken.

```
// The common shortcut – and its real limitations
const cloned = JSON.parse(JSON.stringify(original));
// silently drops: functions, undefined, Symbols
// silently corrupts: Date objects (become strings), Map/Set (become {})
// throws: on circular references

// A real recursive deep clone
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== "object") return value;
  if (seen.has(value)) return seen.get(value); // handles circular refs
  if (value instanceof Date) return new Date(value);

  const clone = Array.isArray(value) ? [] : {};
  seen.set(value, clone);
  for (const key of Object.keys(value)) {
    clone[key] = deepClone(value[key], seen);
  }
  return clone;
}
```

The `JSON.parse(JSON.stringify())` shortcut only round-trips what JSON itself can represent — anything outside JSON's type system (functions, undefined, Symbol, Date, Map/Set) is either silently dropped or silently corrupted into something else. The real implementation walks the object graph recursively (Section 3's reference-vs-value distinction is exactly why a shallow copy — `{...obj}` or `Object.assign` — is insufficient for nested objects), and uses a `WeakMap` to remember already-cloned objects specifically to detect and correctly handle circular references without infinite recursion.

JSON round-trip limitations

WeakMap for circular refs

A directly Glassdoor-reported question. The core idea: since the two trees are structurally identical, a node's position can be described as a **path of child indices** from the root — walk that same path in the second tree to find the corresponding node.

```
function getNodePath(node, root) {
  const path = [];
  while (node !== root) {
    const parent = node.parentNode;
    path.unshift(Array.from(parent.children).indexOf(node));
    node = parent;
  }
  return path;
}

function findCorrespondingNode(node, rootA, rootB) {
  const path = getNodePath(node, rootA);
  let target = rootB;
  for (const index of path) {
    target = target.children[index];
  }
  return target;
}
```

Walking up via `parentNode` and recording each step's child *index* (not tag name or content, which could repeat/vary) builds an unambiguous path back to the root; replaying that same sequence of indices down the second tree lands on the structurally equivalent node — this is a common real building block behind diffing/patching UI libraries when reconciling a virtual tree against corresponding real DOM nodes.

node path by child index

WHAT THE WEB SOURCES EMPHASIZE

Glassdoor reports specifically named "write throttle and/or debounce functions" and "find corresponding nodes between two identical DOM tree structures" as actually-asked coding exercises, not hypothetical practice problems — directly justifying their inclusion here as live-coding questions rather than conceptual ones.

Where language knowledge meets the actual engine running it — hidden classes and generational GC are real, documented V8 mechanisms, not folklore.

8.1 **BASIC** CONCEPT **How can JS leak memory if it has garbage collection?**

Garbage collection automatically frees memory for objects that are no longer **reachable** — reachable from any live root (global variables, the current call stack, closures still in scope). It cannot free memory for an object that's technically still reachable but that the application will genuinely never use again — the GC has no way to know a reference is "logically dead" if something, somewhere, still holds onto it. A memory leak in JS is therefore almost always an **unintentional, lingering reference** — not a language failure, but an application bug where cleanup that should have dropped a reference simply never happened.

reachability

lingering references

8.2 **BASIC** CONCEPT **The most common sources of memory leaks.**

- **Forgotten event listeners** — attaching a listener to a long-lived object (e.g. window) from inside a component that gets destroyed, without ever calling `removeEventListener` — the listener (and everything it closes over, per 1.3) stays reachable through window forever.
- **Detached DOM nodes** — removing a node from the visible document tree via `innerHTML = ""` or similar, while a JS variable somewhere still holds a reference to that now-invisible node — it's fully "detached" from the page but not eligible for collection, since your code still references it.
- **Forgotten timers/intervals** — a `setInterval` that's never `clearInterval`'d keeps its callback (and its closure) alive indefinitely, along with anything that callback references.
- **Closures holding large outer-scope references** — a small, frequently-created closure that happens to close over a reference to a much larger object (1.3's mechanism, unintentionally applied to something big) keeps that large object alive for as long as the small closure survives.

detached DOM nodes

forgotten listeners/timers

Reflow (layout) recomputes the position and size of elements affected by a change — triggered by anything that could alter geometry: changing an element's dimensions, adding/removing DOM nodes, reading certain layout properties (`offsetHeight`, `getBoundingClientRect()`) that force the browser to flush any pending layout work synchronously to answer accurately. **Repaint** only re-draws the pixels for elements whose visual appearance changed (color, visibility) without affecting layout — cheaper, since geometry doesn't need recomputing.

Reflow is more expensive specifically because a layout change can **cascade** — resizing one element can shift the position of many others around it, forcing the browser to recompute geometry for a potentially large portion of the page, not just the one changed element; a repaint, by contrast, is always contained to exactly the visually-changed pixels. The standard mitigation: batch DOM reads and writes separately (read all needed measurements first, then perform all writes) rather than interleaving them, since interleaving forces the browser to repeatedly flush and recompute layout between each read.

[reflow \(layout\)](#)[repaint](#)[layout thrashing](#)

```
// Fast: both objects add properties in the same order → share a hidden class
function Point(x, y) { this.x = x; this.y = y; }
const p1 = new Point(1, 2);
const p2 = new Point(3, 4);

// Slow: different insertion order → different hidden classes, breaks optimization
const p3 = {};
p3.y = 4; p3.x = 3; // y before x — a DIFFERENT shape than Point instances
```

V8 doesn't store object properties in a generic hash map by default — for performance, it dynamically generates internal **hidden classes** (also called "shapes" or "maps") that describe an object's structure, and objects sharing the same hidden class can have their property access compiled to a fast, direct memory offset lookup via **inline caching**, instead of a slower dictionary lookup by name. Every object created with properties added in the *same order* ends up sharing the same hidden class, letting V8 apply this optimization broadly; adding properties in a different order, or adding/deleting properties dynamically after creation, forces V8 to create additional hidden-class transitions (or fall back to a slower, dictionary-mode representation entirely) — a real, measurable performance cost in hot code paths that create many similar objects, which is exactly why consistently initializing all of an object's properties in the constructor, in the same order, is a documented V8 performance idiom rather than a stylistic nitpick.

hidden classes

inline caching

consistent property order

8.5 INTERMEDIATE CONCEPT V8's generational garbage collection.

V8's GC (Orinoco) is built on the **generational hypothesis**: empirically, most objects are extremely short-lived (a temporary variable inside a function call, a small object created and discarded within one event-loop tick), while a much smaller set of objects survive for the life of the application. V8 exploits this by physically separating memory into a small, fast-collected **young generation** (using a cheap, frequent "scavenge" collection) and a larger **old generation** for objects that have survived multiple young-generation collections, collected less often but with more thorough (mark-and-sweep/mark-compact) passes. This two-tier design means the GC spends most of its effort cheaply reclaiming the huge volume of short-lived garbage, and only pays the more expensive full-heap collection cost occasionally, for the comparatively small set of genuinely long-lived objects — directly why 8.2's leak patterns matter: an object that should have died young but is kept reachable gets promoted into the old generation, where it's both more expensive to eventually collect and, until it's identified as a leak, simply never collected at all.

generational hypothesis

young/old generation

Orinoco

8.6 ADVANCED DEBUG

A single-page app gets slower the longer it's open. Diagnose it with the memory profiler.

1. **Take a heap snapshot** in Chrome DevTools' Memory panel at a stable baseline point (e.g. right after initial load).
2. **Perform a repeatable action several times** that's suspected of leaking — navigating between two views, opening and closing a modal — then take a second heap snapshot.
3. **Use the "Comparison" view** between the two snapshots — this specifically highlights objects whose count grew and were *not* garbage collected, rather than requiring you to manually inspect the entire heap.
4. **Look specifically for detached DOM node counts increasing** (8.2) — DevTools explicitly labels these, and a growing count across repeated identical actions is close to a smoking gun.
5. **Inspect the retainer path** for a suspicious growing object — DevTools shows exactly *what* is still holding a reference to it, tracing back to a root (often revealing precisely the forgotten listener or closure from 8.2).

The methodology being tested: this is a repeatable, comparative process (baseline → action → compare), not staring at a single snapshot and guessing — the "Comparison" view and the retainer-path trace are what actually make a leak's root cause findable rather than just confirmable.

heap snapshot comparison

retainer path

- **Debounce the input handler itself** (7.4) — if the expensive work is filtering/re-rendering based on the typed value, there's no reason to redo it on every single keystroke rather than once the user pauses briefly.
- **Avoid unnecessary re-renders of unrelated list items** — in a component framework, this usually means memoizing individual list-item components (keyed correctly) so that only items whose actual data changed re-render, rather than the whole list re-rendering on every parent state update.
- **Virtualize long lists** — render only the subset of items currently visible in the viewport (plus a small buffer), rather than every item in a list that might have thousands of entries; this bounds the DOM node count and reflow cost (8.3) regardless of total data size.
- **Batch DOM writes, read layout properties sparingly** — per 8.3, interleaving reads and writes across many list items forces repeated synchronous reflow; restructure to measure once, then write in one batch.

The unifying interview point: this question deliberately combines earlier sections — debounce (7.4), reflow cost (8.3), and reachability-driven memory behavior (8.1) — and a strong answer explicitly names which specific mechanism each optimization is targeting, rather than offering a vague list of "best practices."

[list virtualization](#)[memoized list items](#)

WHAT THE WEB SOURCES EMPHASIZE

V8-internals deep dives (AppSignal, jayconrod.com, and others) converge specifically on hidden classes plus inline caching and the generational-hypothesis GC design as the two mechanisms worth understanding beyond "V8 is fast" — both used directly, with their actual named mechanisms, in 8.4 and 8.5 rather than generalized "the engine optimizes things" hand-waving.

09 JavaScript Security

The section that connects Section 3's prototype mechanics to real, exploitable vulnerabilities — grounded in Imperva, MDN's own security docs, and Wikipedia's prototype-pollution reference.

9.1

BASIC

SECURITY

What is XSS, and its three common types?

Cross-Site Scripting is a vulnerability class where an attacker gets their own JavaScript to execute in a victim's browser, in the security context of a trusted site — able to read cookies, make authenticated requests, or manipulate the page exactly as the real site's own code could.

- **Stored XSS** — malicious input is saved server-side (a comment, a profile field) and served back to other users later, executing for every victim who views that content.
- **Reflected XSS** — malicious input is included directly in an immediate response (e.g. echoed back from a URL query parameter into an error page), typically requiring a victim to click a crafted link.
- **DOM-based XSS** — the vulnerability lives entirely client-side: JS code takes some attacker-influenced input and unsafely writes it into the DOM (9.2) without ever involving the server round-trip at all.

stored XSS

reflected XSS

DOM-based XSS

9.2

BASIC

SECURITY

Why is innerHTML dangerous, and what's the safer alternative?

```
// Dangerous: if userComment contains "<img src=x onerror=alert(1)>", it EXECUTES
element.innerHTML = userComment;

// Safe: sets it as plain text content, never parsed as HTML/executed
element.textContent = userComment;
```

Assigning to `.innerHTML` parses the string as real HTML and inserts it into the live DOM — any `<script>` tag, event handler attribute (`onerror`, `onclick`), or other executable content embedded in untrusted input runs with full page privileges, the exact mechanism behind DOM-based XSS (9.1). `.textContent` (or, in React, JSX's default text interpolation) inserts the string as literal, inert text — even if it contains characters that look like HTML tags, they're displayed as visible text, never parsed or executed. The rule that generalizes: any API that parses and renders a string as markup is a potential XSS sink whenever the string's content isn't fully trusted, and the safest default is always the text-only API unless rendering real markup is a genuine, deliberate requirement (in which case, sanitize first).

innerHTML XSS sink

textContent

Building directly on 3.7: prototype pollution alone corrupts `Object.prototype`, but that corruption becomes a genuine *exploit*, not just a data-integrity bug, through what it enables downstream. Per the search sources' explicit connection: if an application later checks something like `if (user.isAdmin)` for access control, and `isAdmin` was never set directly on that specific user object, the property lookup falls through the prototype chain (3.1) and finds the polluted property on `Object.prototype` — silently granting elevated access to every single object in the application that doesn't explicitly override that property, an authorization bypass reachable purely through a data-merging bug, with no traditional "vulnerable endpoint" required at all. On the client, per 3.7, a polluted property that later gets rendered into the page (via 9.2's `innerHTML` pattern, for instance) can directly chain into DOM-based XSS.

[prototype pollution → auth bypass](#)[prototype pollution → XSS chain](#)

```
// 1. Freeze the shared prototype – the most direct fix
Object.freeze(Object.prototype);

// 2. Filter dangerous keys explicitly in any merge/clone utility
const DANGEROUS_KEYS = ["__proto__", "constructor", "prototype"];
function safeMerge(target, source) {
  for (const key of Object.keys(source)) {
    if (DANGEROUS_KEYS.includes(key)) continue; // skip them entirely
    target[key] = source[key];
  }
  return target;
}

// 3. Use Map instead of a plain object for untrusted key/value data –
//    a Map has no prototype chain shared with ordinary objects at all
const safeStore = new Map();
```

Per 3.5, `Object.freeze(Object.prototype)` is the blunt, application-wide fix — after freezing, any attempted write to a property on `Object.prototype` silently fails (or throws, in strict mode), closing the vulnerability at its root regardless of how many merge functions exist in the codebase. Explicit key filtering is the more surgical, per-function fix, necessary when a full freeze isn't safe for the application (some libraries legitimately extend built-in prototypes). Using `Map` for any object built from untrusted keys sidesteps the problem structurally — a `Map`'s keys are just entries in an internal data structure, with no relationship to `Object.prototype` at all, so there's no shared chain to pollute in the first place.

`Object.freeze(Object.prototype)``key allowlisting``Map instead of Object`

What is CSP, and how does it limit XSS impact even if a bug exists?

A **Content Security Policy** is a server-sent HTTP header (or meta tag) that restricts what sources a page is allowed to load and execute content from — a defense-in-depth layer, not a replacement for actually fixing XSS bugs (9.2), but a mitigation that limits the *blast radius* when a bug slips through anyway.

```
Content-Security-Policy: script-src 'self' https://trusted-cdn.example.com; object-src 'none'
```

With a strict `script-src` policy in place, even if an attacker successfully injects a `<script>` tag via an XSS vulnerability, the browser itself refuses to execute it unless it comes from an explicitly allowed source — a properly configured CSP can turn a working XSS exploit into a browser console error instead of actual attacker-controlled code execution. A strict policy also commonly disallows inline `<script>` tags and `eval()`-style dynamic code execution entirely, closing off the most common, simplest injection vectors even if some other part of the page has a genuine markup-injection bug.

Content Security Policy

`script-src`

defense in depth

A dependency has a prototype-pollution CVE. Assess impact and mitigate.

1. **Confirm actual exposure** — is the vulnerable merge/clone function in the dependency ever called with data that originates from untrusted input (a request body, query parameters, user-uploaded JSON), or only with fully internal, developer-controlled data? A vulnerable function that's never reachable with attacker-controlled input is a lower-urgency finding than one directly wired to a public API endpoint.
2. **Check the specific downstream impact**, per 9.3 — does this specific application actually read any property (like `isAdmin`-style checks) in a way that would fall through to a polluted `Object.prototype`? The generic CVE severity score doesn't know your application's specific authorization logic; the real risk assessment does.
3. **Apply the immediate mitigation** from 9.4 as a stopgap — `Object.freeze(Object.prototype)` at application startup can neutralize the vulnerability's effect even before the dependency itself is patched, buying time without an urgent, possibly-breaking dependency upgrade.
4. **Upgrade the dependency** to the patched version as the actual fix, and add the specific CVE (or a general prototype-pollution regression test, per 3.7's exploit pattern) to CI so a future dependency downgrade or similar vulnerable pattern introduced elsewhere in the codebase is caught automatically going forward.

exposure assessment before patching

freeze as a stopgap

Cross-Site Request Forgery tricks a victim's browser into making an unwanted, authenticated request to a site the victim is already logged into — e.g. a malicious page silently submits a form to `bank.com/transfer`, and the browser automatically attaches the victim's real session cookies, since cookies are sent based on the target domain, not which page initiated the request. Unlike XSS, the attacker never needs to run any code on the victim's own trusted page at all — the forged request comes from an entirely different, attacker-controlled site.

The **SameSite** cookie attribute is the modern, browser-enforced primary defense: set to `Strict` or `Lax`, it instructs the browser to withhold that cookie on requests originating from a different site, which directly breaks the exact mechanism CSRF depends on — no session cookie attached means no authenticated request, regardless of what a malicious page tries to submit. **CSRF tokens** (a unique, unpredictable value embedded in each legitimate form/request and verified server-side) are the older, complementary defense — even if a cookie somehow gets sent, the attacker's forged request has no way to know or include the correct token, so the server rejects it. Modern practice layers both: SameSite cookies as the default browser-level protection, with explicit CSRF tokens as defense-in-depth for cases (legacy browsers, specific cross-site-cookie needs) where SameSite alone isn't sufficient.

CSRF

SameSite cookies

CSRF tokens

WHAT THE WEB SOURCES EMPHASIZE

Imperva's and Wikipedia's prototype-pollution references, alongside MDN's own security documentation, converge specifically on the pollution-to-XSS chain and `Object.freeze(Object.prototype)` as the standard mitigation — both used directly here rather than generic "sanitize your input" advice.

Sources consulted

Compiled from current MDN, Node.js, and V8-internals documentation, alongside GitHub interview-prep repositories and aggregated practitioner/interview-report sources, as of September 2026. Where this guide says a question is "commonly asked" or "frequently reported," that reflects language or data found directly in the cited source, not an invented popularity claim.

[MDN — In Depth: Microtasks and the JavaScript Runtime Environment](#)

[MDN — JavaScript Prototype Pollution](#)

[Node.js Learn — The Node.js Event Loop, Timers, and process.nextTick\(\)](#)

[Node.js Learn — Backpressuring in Streams](#)

[GitHub — lydiahallie/javascript-questions](#)

[GitHub — GreatFrontEnd Top JavaScript Interview Questions](#)

[GeeksforGeeks JS/DBMS/CN interview-prep hub \(cross-referenced for question framing\)](#)

[Glassdoor — Meta Front End Engineer Interview Questions](#)

[AppSignal Blog — A Deep Dive Into V8](#)

[jayconrod.com — A Tour of V8: Garbage Collection](#)

[Imperva — What Is Prototype Pollution? Risks & Mitigation](#)

[Wikipedia — Prototype Pollution](#)

[DebugBear — How to Optimize Front-end JavaScript Performance](#)

Call Stack Manual · JavaScript / Node.js Edition · Use the "Save as PDF" button (top of page) to export via your browser's print dialog for offline study.